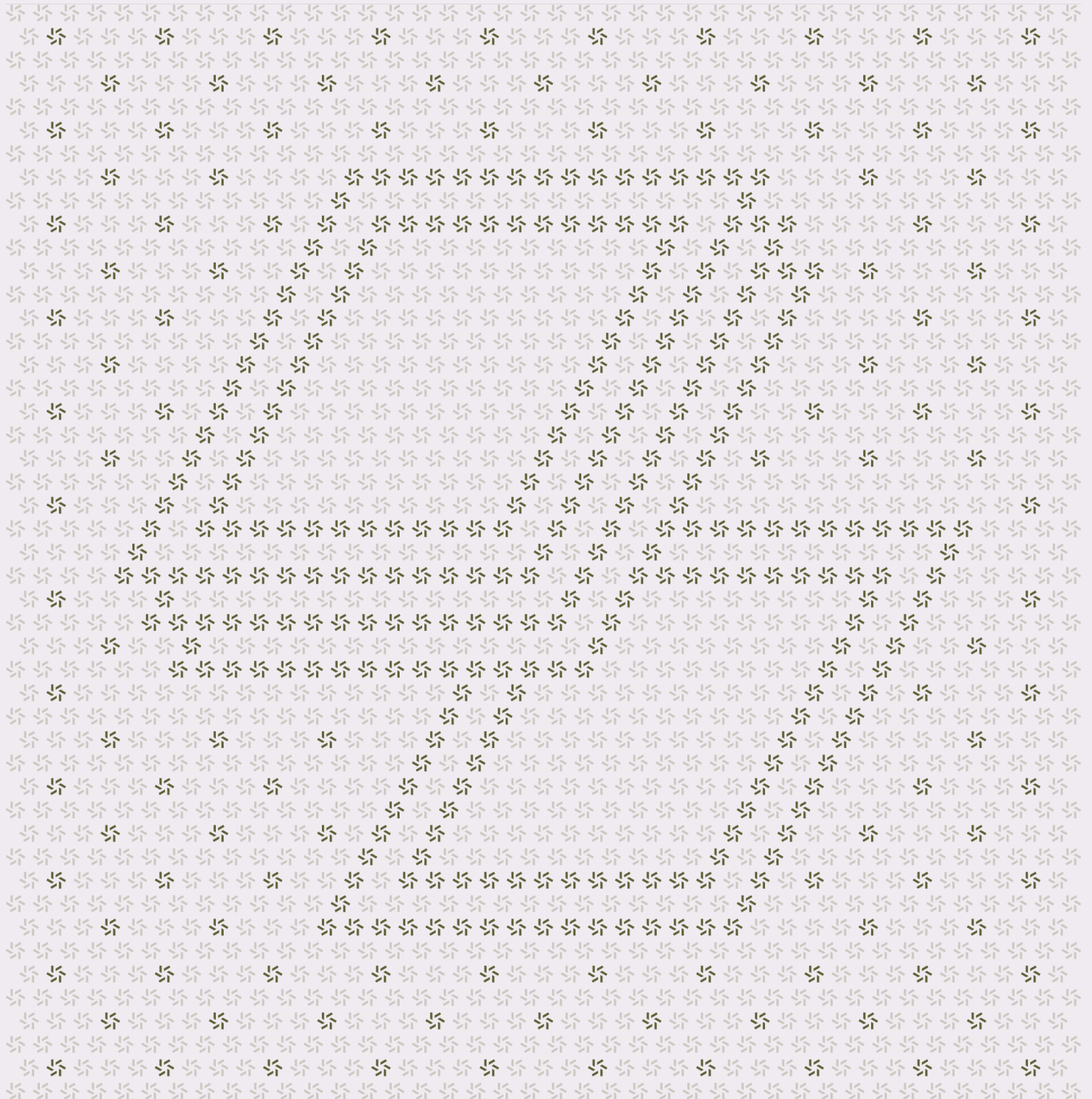


July 13, 2026

librustzcash

Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About librustzcash	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	10
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. ZIP 233 burn amounts can be silently dropped and paid to the miner	11
3.2. An empty witness slice causes a panic in merkle_path_from_slice	14
3.3. An empty transparent bundle serializes identically to an absent one but produces a different txid and sighash	16
3.4. An Orchard change output bypasses the version check and panics while computing a V4 transaction's txid	20
3.5. Building with nu7 but without zip-233 produces invalid V6 transactions	23

4.	System Design	26
4.1.	Component: Orchard and Ironwood Bundle Encoding	26
4.2.	Component: PCZT Verifier Role Adapters	27
4.3.	Component: Signature Hashing and Spend Authorization	28
4.4.	Component: Transaction Builder and Value Balancing	29
4.5.	Component: Transaction Data, Versioning, and Serialization	31
4.6.	Component: Transaction IDs and Authorization Commitments	32

5.	Assessment Results	35
5.1.	Disclaimer	35

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Valar Group from June 22nd to July 2nd, 2026. During this engagement, Zellic reviewed librustzcash's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Could any valid assumptions made by integrators be broken due to errors or issues in this library?
 - Could a transaction built by this library be erroneously modified to cause the user to pay more than they intended?
 - Are there any issues in the serialization / deserialization / parsing code that could cause any integrator codebase to crash or panic?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

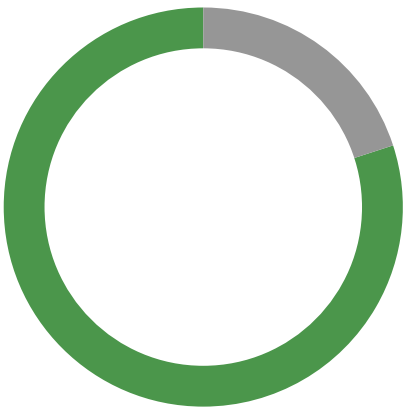
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped librustzcash targets, we discovered five findings. No critical issues were found. Four findings were of low impact and the remaining finding was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	4
Informational	1



2. Introduction

2.1. About librustzcash

Valar Group contributed the following description of librustzcash:

Librustzcash is responsible for transaction parsing, and signature hashing for Zcash full nodes. This audit covers those consensus functionalities, that the updates to v6 transaction parsing and signature hashing are safe. The changes here are relatively small, as v6 includes an ironwood bundle that is nearly identical to Orchard's. Signature hashing is following the same format as existing signing as well, with the updated design detail of removing signing over the anchor hashes.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the targets.

Architecture risks. This encompasses potential hazards originating from the blueprint of a system, which involves its core validation mechanism and other architecturally significant constituents influencing the system's fundamental security attributes, presumptions, trust mode, and design.

Arithmetic issues. This includes but is not limited to integer overflows and underflows, floating-point associativity issues, loss of precision, and unfavorable integer rounding.

Implementation risks. This encompasses risks linked to translating a system's specification into practical code. Constructing a custom system involves developing intricate on-chain and off-chain elements while accommodating the idiosyncrasies and challenges presented by distinct programming languages, frameworks, and execution environments.

Availability. Denial-of-service attacks are another leading issue in custom systems. Issues including but not limited to unhandled panics, unbounded computations, and incorrect error handling can potentially lead to consensus failures.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

librustzcash Targets

Type	Rust
Platform	Zcash
Target	Only changes between 412a91...5df7f0
Repository	https://github.com/valargroup/librustzcash ↗
Version	5df7f06486c7c1cab317a069809b7147857e37ee
Programs	pczt/src/roles/verifier/orchard.rs pczt/src/roles/verifier/sapling.rs pczt/src/roles/verifier/transparent.rs zcash_primitives/src/transaction/builder.rs zcash_primitives/src/transaction/mod.rs zcash_primitives/src/transaction/sighash.rs zcash_primitives/src/transaction/sighash_v5.rs zcash_primitives/src/transaction/sighash_v6.rs zcash_primitives/src/transaction/txid.rs zcash_primitives/src/transaction/components/orchard.rs
Target	orchard
Repository	https://github.com/zcash/orchard ↗
Version	69346a6f26768aa23172bd6437995fde60975da9
Programs	bundle/commitments.rs pczt/parse.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.7 person-weeks. The assessment was conducted by three consultants over the course of one calendar week.

Contact Information

The following project manager was associated with the engagement:

 **Chad McDonald**
Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

 **Syed Faraz Abrar**
Security Engineer
faith@zellic.io ↗

 **Ingab Kang**
Security Engineer
ingab@zellic.io ↗

 **ZhiHan Wei**
Security Engineer
zhihan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 22, 2026 Start of primary review period

July 2, 2026 End of primary review period

3. Detailed Findings

3.1. ZIP 233 burn amounts can be silently dropped and paid to the miner

Target	builder.rs, mod.rs		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The transaction builder and the TransactionData constructors allow a ZIP 233 burn amount to be set on a transaction version whose wire format has no field to carry it. Neither path rejects this mismatch, so the burn is counted in the value balance, meaning the inputs must fund it, yet it is never serialized. The value intended to be burned is instead released to the miner as fee, and the fee reported by fee_paid is smaller than the amount that actually leaves the sender on chain.

The Builder stores the burn in its zip233_amount field, which is set through set_zip233_amount. At build time, check_version_compatibility is responsible for rejecting any component that the chosen version cannot represent. It rejects Sapling, Orchard, and Ironwood components, but it never inspects zip233_amount:

```
fn check_version_compatibility<FE>(&self, version: TxVersion) -> Result<(),
    Error<FE>> {
    if !version.valid_in_branch(self.consensus_branch_id) {
        return Err(Error::TargetIncompatible(
            self.consensus_branch_id,
            version,
            None,
        ));
    }

    let sapling_available = version.has_sapling()
    && self.consensus_branch_id.has_sapling();
    if !sapling_available
        && (!self.sapling_inputs().is_empty()
        || !self.sapling_outputs().is_empty())
    {
        return Err(Error::TargetIncompatible(
            self.consensus_branch_id,
            version,
            Some(PoolType::SAPLING),
        ));
    }
}
```

```
// [...]  
  
Ok()  
}
```

The burn is nevertheless included in the balance calculation: `value_balance` adds a negative burn term, `-ZatBalance::from(self.zip233_amount)`, so the transaction inputs must cover it.

ZIP 233 is only supported from V6 onward, and `TxVersion::has_zip233` returns `false` for V5. A V5 transaction is still valid in the NU6.3 and NU7 branches, as `valid_in_branch` returns `true` for those cases, so a caller can legitimately select V5 while a burn is configured. When such a transaction is serialized, `write_v5` and its `write_v5_header` helper do not emit the burn, because only `write_v6_header` writes it. The amount is therefore silently discarded.

The same missing validation exists in `TransactionData::from_parts`, which accepts a `zip233_amount` argument and stores it without comparing it against `version.has_zip233()`. Constructing a `TransactionData` with `TxVersion::V5`, `BranchId::Nu7`, and a nonzero `zip233_amount` reproduces the same inconsistency directly at the `TransactionData` level.

A related gap exists on the Partially Created Zcash Transaction (PCZT) path. `build_for_pczt` consumes a `Builder` that may hold a nonzero `zip233_amount` and includes it in the same balance check, but the `PcztParts` structure it returns has no field to carry the burn:

```
/// The components of a PCZT.  
#[derive(Debug)]  
pub struct PcztParts<P: Parameters> {  
    pub params: P,  
    pub version: TxVersion,  
    pub consensus_branch_id: BranchId,  
    pub lock_time: u32,  
    pub expiry_height: BlockHeight,  
    pub transparent: Option<transparent::pczt::Bundle>,  
    pub sapling: Option<sapling::pczt::Bundle>,  
    pub orchard: Option<orchard::pczt::Bundle>,  
    #[cfg(zcash_unstable = "nu6.3")]  
    pub ironwood: Option<orchard::pczt::Bundle>,  
}
```

As a result, any burn configured before `build_for_pczt` is silently dropped and cannot be conveyed to later PCZT roles. If ZIP 233 burns are meant to be expressible through a PCZT, `PcztParts` currently provides no way to preserve the amount across that boundary.

Impact

Configuring a burn on a version that cannot serialize it produces an economically incorrect transaction that is otherwise valid and spendable. The value intended to be burned is paid to the

miner as additional fee, because the builder requires the inputs to fund the burn while the serialized transaction contains none. Because `fee_paid` subtracts `zip233_amount` from its result, it reports a fee lower than the amount the sender actually forfeits on chain, so any wallet relying on it would display an inaccurate fee. On the PCZT path, a configured burn is discarded, so a transaction later assembled from the PCZT does not perform the intended burn.

No external party can trigger this. It arises only when the transaction is deliberately built with a burn on a version that cannot serialize it, so the funds at stake are always the sender's own.

Recommendations

The inconsistent configuration should be rejected as early as possible, mirroring the existing per-pool checks. The most targeted place is `check_version_compatibility`, which should return `Error::TargetIncompatible` when `self.zip233_amount != Zatoshis::ZERO` and `!version.has_zip233()`, covering every transaction produced through the builder. The same invariant should also be enforced at serialization. Because `write_v5`, through `write_v5_header`, already returns an `io::Result`, it can fail when `zip233_amount` is nonzero, which is a reliable catch-all that rejects an inconsistent transaction regardless of how it was constructed. The constructor `TransactionData::from_parts` is infallible, since it returns `Self`, and so cannot report the mismatch as an error; it should either be given a fallible variant that validates `zip233_amount` against `has_zip233` or document the precondition, with the serialization check acting as the backstop.

If ZIP 233 burns are intended to be carried through a PCZT, add a `zip233_amount` field to `PcztParts` and thread it through `build_for_pczt` and the PCZT roles so that the amount is preserved rather than discarded.

Corresponding tests should also be added, including a nonzero burn paired with a version for which `has_zip233` returns `false`, and a `build_for_pczt` call that configures a burn.

Remediation

This issue has been acknowledged by Valar Group.

Valar Group provided the following comment:

This issue was not applicable in production. It was caused by incorrect build flags in the version of the code provided for the audit. We will verify that the production build flags are correct

3.2. An empty witness slice causes a panic in merkle_path_from_slice

Target	merkle_tree.rs		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The public function `merkle_path_from_slice` decodes a Merkle path from its serialized form and is exported from the `zcash_primitives` crate. It returns an `io::Result`, so malformed input is expected to be surfaced as an `Err`. Before validating the length of its input, however, it indexes the first byte of the witness slice directly:

```
/// Reads a Merkle path from its serialized form.
pub fn merkle_path_from_slice<Node: HashSer, const DEPTH: u8>(
    mut witness: &[u8],
) -> io::Result<MerklePath<Node, DEPTH>> {
    // Skip the first byte, which should be DEPTH to signify the length of
    // the following vector of Pedersen hashes.
    if witness[0] != DEPTH {
        return Err(io::Error::new(
            io::ErrorKind::InvalidData,
            "depth is not as expected",
        ));
    }
    // [...]
}
```

When `witness` is empty, the expression `witness[0]` indexes past the end of the slice and panics because the index is out of bounds, instead of returning an `Err`. Every other malformed input is reported as an `io::Error`, whether an unexpected depth byte, an auth path element of the wrong length, an auth path of the wrong size, or trailing data. The empty slice is therefore the only input that aborts rather than failing gracefully. The remaining accesses to each element are bounded by `chunks_exact(33)`, which yields chunks of a fixed size, so the initial index is the only unchecked one.

The function carries no documentation stating that the witness must be nonempty, and it has no internal production callers within the crate; its only uses are in tests. It is therefore exposed as an API for external integrators, any of which could reach this panic by forwarding an empty or otherwise attacker-supplied slice.

Impact

Indexing past the end of a slice in Rust panics. Depending on how the calling code is configured, an unhandled panic unwinds or aborts the thread, so a consumer that decodes untrusted bytes through this function can be made to crash on a trivially malformed input, resulting in a denial of service. The panic is a liveness problem, not one of memory safety, and no funds are at risk. Because the crate itself never calls the function outside tests, whether the bug can be reached at all depends on how an integrator wires it up.

Recommendations

Validate that the `witness` slice is nonempty before indexing it, and return an `io::Error` consistent with the function's other failure paths. Using `split_first` performs the check and avoids the direct index at once:

```
let (depth_byte, rest) = witness.split_first().ok_or_else(|| {
    io::Error::new(io::ErrorKind::InvalidData, "witness is empty")
})?;
if *depth_byte != DEPTH {
    return Err(io::Error::new(
        io::ErrorKind::InvalidData,
        "depth is not as expected",
    ));
}
witness = rest;
```

Documenting the input expectations of the public function is also advisable. Because the function may receive untrusted input, however, returning an error is the robust fix rather than relying on documentation alone.

Remediation

This issue has been acknowledged by Valar Group.

3.3. An empty transparent bundle serializes identically to an absent one but produces a different txid and sighash

Target	txid.rs		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The txid and sighash computations decide whether a transparent bundle is present by testing whether the `Option` is `Some`, rather than by testing whether the bundle actually holds any inputs or outputs. As a result, a bundle that is present but empty, meaning `Some` with both `vin` and `vout` empty, is committed to as a real transparent component, whereas an absent bundle, `None`, is committed to as nothing. The serializer, by contrast, writes both forms identically, and the parser maps the resulting bytes back to `None`, so the two forms produce the same serialized transaction while computing different txids and shielded sighashes.

The txid digester derives its transparent digest in two steps, neither of which treats an empty bundle as absent:

```
fn digest_transparent(
    &self,
    transparent_bundle: Option<&transparent::Bundle<A::TransparentAuth>>,
) -> Self::TransparentDigest {
    transparent_bundle.map(transparent_digests)
}

pub(crate) fn hash_transparent_txid_data(
    t_digests: Option<&TransparentDigests<Blake2bHash>>,
) -> Blake2bHash {
    let mut h = hasher(ZCASH_TRANSPARENT_HASH_PERSONALIZATION);
    if let Some(d) = t_digests {
        h.write_all(d.prevouts_digest.as_bytes()).unwrap();
        h.write_all(d.sequence_digest.as_bytes()).unwrap();
        h.write_all(d.outputs_digest.as_bytes()).unwrap();
    }
    h.finalize()
}
```

Because `hash_transparent_txid_data` absorbs three component digests, each 32 bytes, when the digest is `Some` and nothing when it is `None`, the two cases finalize to different values, even though the component digests of an empty `vin` and `vout` are themselves fixed.

The serializer and parser, however, treat the two forms as equivalent:

```
pub fn write_transparent<W: Write>(&self, mut writer: W) -> io::Result<()> {
    if let Some(bundle) = &self.transparent_bundle {
        Vector::write(&mut writer, &bundle.vin, |w, e| e.write(w))?;
        Vector::write(&mut writer, &bundle.vout, |w, e| e.write(w))?;
    } else {
        CompactSize::write(&mut writer, 0)?;
        CompactSize::write(&mut writer, 0)?;
    }

    Ok(())
}

fn read_transparent<R: Read>(
    mut reader: R,
) -> io::Result<Option<transparent::Bundle<transparent::Authorized>>> {
    let vin = Vector::read(&mut reader, TxIn::read)?;
    let vout = Vector::read(&mut reader, TxOut::read)?;
    Ok(if vin.is_empty() && vout.is_empty() {
        None
    } else {
        Some(transparent::Bundle {
            vin,
            vout,
            authorization: transparent::Authorized,
        })
    })
}
```

An empty transparent bundle therefore serializes to two vectors of zero length, exactly like None, and read_transparent canonicalizes those bytes back to None. The canonical representation on chain of an empty transparent bundle is thus None.

The shielded pools do not share this problem. In to_txid, an absent Sapling, Orchard, or Ironwood bundle is committed to using the empty digest defined by the protocol. For Sapling this is sapling_digest.unwrap_or_else(hash_sapling_txid_empty), with the equivalent for Orchard and Ironwood. An empty shielded bundle hashes to that same empty digest because the shielded digest helpers skip writing when the bundle has no spends or outputs. A present but empty shielded bundle and an absent one therefore commit to the same value. The transparent digest has no equivalent canonicalization.

The same divergence reaches the signature hashes. Both v5_signature_hash and v6_signature_hash derive their transparent digest from the same transparent_digests value, and transparent_sig_digest branches on whether it is present, so a present but empty bundle changes the shielded sighash that Sapling, Orchard, and Ironwood spends sign. Such a bundle has no inputs of its own, so no transparent input signatures are involved.

The transaction builder does not produce this state, because the transparent builder, in the `zcash_transparent` crate, collapses an empty bundle to `None` before it reaches `TransactionData`. The inconsistency is reachable only through the constructors such as `TransactionData::from_parts`, which store the caller's `Option` without collapsing an empty bundle.

Impact

A transaction whose state in memory holds a present but empty transparent bundle commits to a `txid` and signature hashes that do not correspond to its own canonical serialization, which reparses the bundle as `None`. The `txid` computed from the transaction in memory does not match the `txid` computed from the serialized bytes, so the transaction may be referenced or tracked under an identifier that does not exist on chain. Because the shielded sighash commits to the transparent digest, the signatures of any Sapling, Orchard, or Ironwood spends are produced over a preimage that does not match the serialized transaction, so a transaction that carries such spends fails signature validation once serialized.

Reaching this state at all requires one of those constructors, because the standard builder collapses an empty bundle before it can arise. A transaction that does reach it computes a `txid` that its own serialized bytes never reproduce, and if it also carries shielded spends, those signatures fail the moment it is serialized. The fallout therefore stays with whoever assembled the transaction through that path; funds are safe and consensus is unaffected.

Recommendations

Ensure that a present but empty transparent bundle is never distinguished from an absent one, matching the serialized representation and the handling of the shielded pools. The most robust fix is to canonicalize the bundle at construction, so that the inconsistent state in memory cannot be formed: have `TransactionData::from_parts`, and the other constructors that accept a transparent bundle, collapse an empty bundle to `None`, mirroring what the transparent builder already does. For example, in `from_parts`:

```
transparent_bundle: transparent_bundle
    .filter(|b| !(b.vin.is_empty() && b.vout.is_empty())),
```

With this change, the `txid`, both sighash versions, and serialization all treat an empty bundle as absent, and nothing else needs to be modified.

Collapsing the bundle only inside `digest_transparent` is not sufficient on its own, because `v5_signature_hash` and `v6_signature_hash` each begin by asserting that the transparent bundle and its digest are either both present or both absent:

```
assert_eq!(
    tx.transparent_bundle.is_some(),
    txid_parts.transparent_digests.is_some())
```

```
);
```

Collapsing the digest to None while leaving the bundle as Some would fail this assertion and panic the signing path, so a fix at the digest layer would also have to relax the assertion and adjust how each function zips the bundle with its digest. Canonicalizing at construction avoids all of this.

A regression test should assert that a transaction carrying a present but empty transparent bundle and the same transaction carrying None compute identical txids and sighashes, and that serializing and reparsing either one leaves it unchanged.

Remediation

This issue has been acknowledged by Valar Group.

3.4. An Orchard change output bypasses the version check and panics while computing a V4 transaction's txid

Target	builder.rs		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

`check_version_compatibility` is responsible for rejecting a transaction version that cannot represent the pooled inputs and outputs already added to the builder. Its first Orchard guard tests only whether the Orchard builder has spends or outputs; it does not consider the change set:

```
let orchard_available = version.has_orchard()
    && self.consensus_branch_id.has_orchard();
if !orchard_available
    && self
        .orchard_builder
        .as_ref()
        .is_some_and(|b| !b.spends().is_empty() || !b.outputs().is_empty())
{
    return Err(Error::TargetIncompatible(
        self.consensus_branch_id,
        version,
        Some(PoolType::ORCHARD),
    ));
}
```

A change output leaves `spends()` and `outputs()` empty while populating `changes()`, so this guard does not fire when the Orchard builder holds only a change. The second Orchard guard does not catch it either. Although `orchard_in_use` includes the change set, the guard also requires `!orchard_protocol_matches_version(version)`, and the protocols match in this case. The function `orchard_protocol_matches_version` compares the builder's protocol against `orchard_protocol_for_version`, which for V4 resolves through `orchard_protocol_for_branch` to the branch's protocol. On the NU6.3 branch, that protocol is `OrchardPostNu6_3`, the same protocol the builder already holds, so the guard is skipped.

As a result, a V4 transaction built on the NU6.3 branch with an Orchard change output passes validation. The build then places the Orchard bundle, which was built for the `OrchardPostNu6_3` protocol, into a V4 transaction and computes its txid. In `digest_orchard`, part of the txid computation, the bundle is committed using the domain for the transaction version:

```
fn digest_orchard(
    &self,
    version: TxVersion,
    orchard_bundle: Option<&orchard::Bundle<A::OrchardAuth, ZatBalance>>,
) -> Self::OrchardDigest {
    orchard_bundle.map(|b| {
        b.commitment(orchard_commitment_domain(version))
            .expect("Orchard bundle flags must be representable in their
transaction format")
            .0
    })
}
```

For V4, `orchard_commitment_domain` returns `ORCHARD_V5_PRE_NU6_3`, which cannot represent the flags of an `OrchardPostNu6_3` bundle. The commitment therefore returns an error, and the `expect` call converts it into a panic rather than a clean failure.

Impact

The transaction build panics instead of returning an error, crashing the process that constructs the transaction and constituting a denial of service for a caller that reaches this configuration. The panic occurs during construction, is not triggered by an attacker on a validating node, and does not lead to memory unsafety or loss of funds, which limits its severity.

Recommendations

Make the first Orchard guard account for change outputs, consistent with `orchard_in_use`, so that Orchard content of any kind is rejected on a version that cannot represent it. Reusing the existing helper covers all three sets at once:

```
let orchard_available = version.has_orchard()
    && self.consensus_branch_id.has_orchard();
if !orchard_available && self.orchard_in_use() {
    return Err(Error::TargetIncompatible(
        self.consensus_branch_id,
        version,
        Some(PoolType::ORCHARD),
    ));
}
```

With this change, a V4 transaction carrying any Orchard spends, outputs, or changes is rejected before the incompatible bundle reaches the digest.

As defense in depth, `digest_orchard` could surface an unrepresentable bundle as an error rather

than panicking through the `expect` call, so that a version and bundle mismatch reached by any path fails gracefully.

A test should also cover an Orchard builder that holds only a change and is proposed as a version without Orchard support, asserting that the build returns `Error::TargetIncompatible` rather than panicking.

Remediation

This issue has been acknowledged by Valar Group.

3.5. Building with nu7 but without zip-233 produces invalid V6 transactions

Target	Project-wide		
Category	Code Maturity	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The TxVersion::V6 variant is enabled, and treated as valid for the NU7 consensus branch, by the zcash_unstable = "nu7" configuration alone. In valid_in_branch, the V6 arm and its NU7 case are gated only on that configuration:

```
#[cfg(any(zcash_unstable = "nu6.3", zcash_unstable = "nu7"))]
TxVersion::V6 => match consensus_branch_id {
    Sprout | Overwinter | Sapling | Blossom | Heartwood | Canopy | Nu5 | Nu6
    | Nu6_1 | Nu6_2 => false,
    #[cfg(zcash_unstable = "nu6.3")]
    Nu6_3 => true, // Ironwood / NU6.3
    #[cfg(zcash_unstable = "nu7")]
    Nu7 => true, // ZIP 230 or ZIP 248, whichever is chosen for activation
    #[cfg(zcash_unstable = "zfuture")]
    ZFuture => false,
},
```

The ZIP 233 burn amount that a V6 header carries under NU7, however, is gated behind an additional Cargo feature, zip-233. Its zip233_amount field is read only when the feature and a configuration that carries the burn are both present, as in read_v6_header_fragment:

```
fn read_v6_header_fragment<R: Read>(mut reader: R) ->
io::Result<V6HeaderFragment> {
    let (consensus_branch_id, lock_time, expiry_height) =
        Self::read_header_fragment(&mut reader)?;

    Ok(V6HeaderFragment {
        consensus_branch_id,
        lock_time,
        expiry_height,
        #[cfg(all(
            any(zcash_unstable = "nu7", zcash_unstable = "zfuture"),
            feature = "zip-233"
        ))]
    })
```

```
        zip233_amount: Self::read_zip233_amount(&mut reader)?,
    })
}
```

The write of `zip233_amount` within `write_v6_header`, and the `has_zip233` helper, are gated by the same `all(any(zcash_unstable = "nu7", zcash_unstable = "zfuture"), feature = "zip-233")` condition, so the burn amount is written, read, and committed to only when the feature is enabled.

These are two independent build mechanisms. The configuration `zcash_unstable = "nu7"` is supplied as a `--cfg` compiler flag through `RUSTFLAGS`, whereas the `zip-233` Cargo feature is declared in `Cargo.toml`. A `--cfg` flag cannot enable a Cargo feature, and no `compile_error!` guard requires the two together. A build that sets `zcash_unstable = "nu7"` but omits the `zip-233` feature therefore compiles cleanly, yet its V6 header contains no burn amount.

Impact

On a conformant NU7 network, where the burn amount is part of the V6 header format, a transaction produced by a build with `nu7` but not `zip-233` is malformed, since its header is 8 bytes short, and would be rejected by peers. Conversely, such a build cannot parse conformant V6 transactions. The mismatch is silent: the build succeeds, and the library is internally consistent because it neither writes nor reads the field, so the problem surfaces only as invalid transactions when the library interacts with a correctly configured network.

The trigger is a combination of build flags set at compile time, not a runtime input, so the fault lies in how the library is packaged. In practice it would be caught the first time such a build exchanged transactions with a conformant peer, since the malformed header would fail to parse.

Recommendations

Enforce the coupling at compile time. Because `zcash_unstable = "nu7"` cannot enable a Cargo feature, add a `compile_error!` guard, mirroring the existing gate, that fails the build when a configuration carrying the burn is selected without the `zip-233` feature:

```
#[cfg(all(
    any(zcash_unstable = "nu7", zcash_unstable = "zfuture"),
    not(feature = "zip-233")
))]
compile_error!(
    "building with `zcash_unstable=\"nu7\"` or `\"zfuture\"` requires the\n`zip-233` feature"
);
```

Documenting the required flag combination is also worthwhile, but enforcing it at compile time is preferable because it makes the invalid configuration impossible rather than relying on the reader

to remember it. If `zip-233` is instead intended to be toggled independently under `nu7`, the validity and header handling of `V6` should be gated so that they remain consistent when the feature is absent.

Remediation

This issue has been acknowledged by Valar Group.

Valar Group provided the following comment:

This issue was not applicable in production. It was caused by incorrect build flags in the version of the code provided for the audit. We will verify that the production build flags are correct

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Component: Orchard and Ironwood Bundle Encoding

Description

This component is responsible for serializing Orchard transaction bundles. It is implemented in `zcash_primitives/src/transaction/components/orchard.rs`.

The component reads and writes the following into transaction bundles:

- Action data
- Bundle flags
- Balances
- Anchors
- Proofs
- Spend authorization signatures
- Binding signatures

Invariants

- V5 and V6 bundle flag bytes must be interpreted under the correct `BundleFormat`.
- Invalid data (such as commitments, anchors, nullifiers, etc) must be rejected during parsing.
- Proof byte lengths must be canonical for the number of actions.
- V6/NU6.3 bundle serialization must reject flags that cannot be represented in the selected transaction format.
- Orchard and Ironwood bundles must not be confused even though both use `orchard::Bundle`; higher layers must supply the correct protocol/domain.
- Empty bundles must serialize as zero actions and parse as `None`.

Test coverage

Cases covered

- Orchard V5 and V6 bundle read/write code paths are tested.
- Proof size enforcement using `ProofSizeEnforcement::Strict` is tested.
- Bundle flag parsing under pre-NU6.3 and NU6.3 formats is tested.

- Orchard bundle generation and serialization behavior are both tested.

Cases not covered

- Malformed flag bytes for both PreNu6_3 and Nu6_3 formats are not tested.
- Mismatched Orchard bundle supplied where Ironwood bundle is expected, or vice versa, is not tested.
- DOS / out-of-memory conditions are not tested.

Attack surface

- Transaction parsers consume attacker-controlled bytes from the network.
- Bundle flags influence whether spends, outputs, and cross-address transfers are enabled.
- Extremely large action counts or vectors can lead to resource exhaustion.
- Low-level callers can construct invalid Orchard bundle objects directly and then request serialization.
- Misinterpreting the NU6.3 flag byte could permit invalid transfers or reject valid consensus transactions.

4.2. Component: PCZT Verifier Role Adapters

Description

This component is responsible for providing parsed PCZT (Partially Created Zcash Transaction) bundles to the caller. It is implemented in `pczt/src/roles/verifier/orchard.rs`, `sapling.rs`, and `transparent.rs`.

Each verifier method takes a `Pczt`, parses one bundle, then calls a caller-supplied closure with this bundle. It then serializes the bundle back into the PCZT if the closure succeeds.

This component is not a complete verifier. It provides integrating protocols a way to run checks using the PCZT bundle APIs. The actual verification performed depends on the caller-supplied closure.

Invariants

- The provided bundle must parse successfully before the caller-supplied closure is invoked.
- Orchard bundles must be parsed using the bundle format specified by the PCZT global fields.
- Ironwood bundles must be parsed as Ironwood.

Test coverage

Cases covered

- Ironwood methods rejecting V5 PCZTs before invoking callbacks is tested.
- Ironwood validation through PCZT parsing and role entry points is tested.
- Orchard verifier adapter parsing and serialization are tested.

Cases not covered

- End-to-end tests where verifier closures reject malicious bundle values, recipients, or proofs are not present.
- Tests for malicious proprietary fields or redacted metadata that could affect the signer UX are not present.
- Full V6/ZIP 233 PCZT workflow tests are not present.

Attack surface

- PCZT bytes may come from untrusted sources.
- Integrators may misunderstand the role name and assume the adapter performs proof, value, recipient, or ownership verification automatically.
- Ironwood-specific entry points depend on global version and branch fields, which are attacker-controlled in the PCZT until parsed and checked.
- Redacted PCZT data can limit what a verifier or signer can validate.

4.3. Component: Signature Hashing and Spend Authorization

Description

This component is responsible for computing the digests that Sapling, Orchard, Ironwood, and TZE spend authorizations sign. It is implemented in `zcash_primitives/src/transaction/sighash.rs`, `sighash_v5.rs`, and `sighash_v6.rs`.

`signature_hash` dispatches to the correct version-specific sighash function based on the transaction version. `v5_signature_hash` implements the ZIP 244-style signature digest used for V5. `v6_signature_hash` is the V6 entry point.

Invariants

- Every spend signature must commit to exactly the transaction data required by the active transaction version and sighash type.
- Transparent input indices must refer to real transaction inputs.
- Shielded Sapling, Orchard, and Ironwood signatures in the same transaction must sign the intended shared shielded sighash.

- TZE sighash data must only be constructed for transactions that support TZE.

Test coverage

Cases covered

- V4 and V5 transparent sighash related tests exist.
- ZIP 233 V6 shielded sighash related tests exist.
- Construction of valid transparent `SignableInput` values through the transparent crate helper is tested.
- V6 txid/auth commitment behavior is tested.

Cases not covered

- Tests for V6 transparent sighash vectors for all sighash flag combinations are not present.
- Tests where invalid transparent input indices reaching `v5_signature_hash` or `v6_signature_hash` are not present.
- Tests for Invalid TZE input indices are not present.
- Tests for PCZT V6 sighash behavior are not present.
- Explicit tests where changing unsupported in-memory fields does not affect signatures for versions that cannot serialize those fields are not present.

Attack surface

- Wallets and hardware signers rely on the sighash to represent exactly what the user is authorizing.
- Attackers may attempt transaction mutation where a changed field is not covered by the relevant signature hash.
- Transparent sighash flags intentionally allow partial signing. This can lead to authorizing more than the user intended.
- Indices in signing inputs are caller controlled and can cause panics or incorrect authorization if not validated before hashing.
- The V6 transaction format is high risk because it may share code with V5 while making modifications to fields or other rules.

4.4. Component: Transaction Builder and Value Balancing

Description

This component is responsible for constructing transactions from inputs provided by the caller. These inputs include spends, outputs, target height, fee rules, and other signing inputs. It is implemented in `zcash_primitives/src/transaction/builder.rs`.

The transaction builder selects the transaction version and shielded bundle protocols for the specified consensus branch, and then exposes methods for adding transparent / Sapling / Orchard / Ironwood spends and outputs. It then calculates required fees, checks balances, creates proofs, applies signatures, and returns either a final `Transaction`, or `PCZT` parts.

Separate builders are maintained per pool. Transparent, Sapling, Orchard, and Ironwood each accumulate their own inputs and outputs. At build time, their balances are summed together with any other values (such as ZIP 233 burns). For shielded spends, the builder computes a shared shielded sighash and applies Sapling, Orchard, and Ironwood signatures before freezing the transaction.

Invariants

- The selected transaction version must be valid for the selected consensus branch.
- A transaction version must not be allowed to carry fields or bundles that its wire format cannot represent.
- The sum of transparent, Sapling, Orchard, Ironwood, ZIP 233, TZE, and fee values must cancel out to zero before a transaction is produced.
- Orchard and Ironwood versions must be separated.
- All shielded spends in the transaction must be authorized by signatures over the intended sighash.
- PCZT construction must preserve every field that was included in the builder's balance checks, or reject unsupported fields.

Test coverage

Cases covered

- Transaction building with transparent, Sapling, and Orchard components - has existing tests.
- Fee and insufficient-funds handling for ordinary outputs and ZIP 233 accounting paths - has existing tests.
- Rejection of Orchard outputs when cross-address transfers are disabled - has existing tests.
- Acceptance of Orchard change outputs when cross-address transfers are disabled - has existing tests.
- Rejection of wrong note versions for Orchard and Ironwood spends - has existing tests.
- PCZT construction preserves V6 and rejects V5 when Ironwood is used - has existing tests.

Cases not covered

- Non-zero ZIP 233 amount with a transaction version that does not support ZIP 233 - no existing tests.
- PCZT construction with nonzero ZIP 233 amount - no existing tests.

- End-to-end test for an Ironwood -> Orchard transfer that enforces that the same account key is used - no existing tests.

Attack surface

- Callers control the target height, proposed transaction version, expiry height, spends, outputs, recipients, values, memos, anchors, fee rule, and ZIP 233 burn amount.
- Fee rules and balance logic are security-sensitive because they decide whether funds become outputs, burns, or miner fees.
- PCZT output is an intermediate output, therefore any field omitted at this boundary can change what later signers authorize.

4.5. Component: Transaction Data, Versioning, and Serialization

Description

This component is responsible for representing transaction data in memory and converting it to and from consensus byte encodings. It is implemented in

`zcash_primitives/src/transaction/mod.rs`.

It defines `TxVersion`, `TransactionData`, and `Transaction`. `TxVersion` describes which transaction versions are valid in which consensus branches and which protocol features each version supports. `TransactionData` stores the version, branch ID, lock time, expiry height, pool bundles, and any other version-specific fields. `Transaction` wraps authorized data with a computed transaction ID.

This component parses transactions by reading the version first and then dispatching to version-specific readers. It serializes transactions by dispatching to `write_v4()`, `write_v5()`, or `write_v6()`. It also exposes low-level constructors such as `TransactionData::from_parts()`, `from_parts_v6()`, and `from_parts_zfuture()`, and computes fee accounting over the in-memory transaction state.

Invariants

- `TxVersion::has_*` capability helpers must match the fields that each version can carry.
- In-memory transaction data must not contain nonzero or nonempty fields that the selected transaction version cannot serialize.
- Version-specific read and write functions must consume and produce the exact same byte layout.
- Parsing untrusted bytes must reject invalid encodings, invalid points, invalid values, and unsupported versions.
- `TransactionData::freeze()` must not turn invalid in-memory state into a transaction whose serialized form has different economic meaning.
- Fee accounting must match the economic effects represented by the selected

version's wire format.

Test coverage

Cases covered

- Transaction serialization roundtrips for several consensus branches - has existing tests.
- V6/ZIP 233 test vectors for transaction ID, authorization commitment, and shielded sighash - has existing tests.
- Rejection of Orchard components in V4 serialization - has existing tests.
- Rejection of Sprout components in V5/V6 serialization - has existing tests.

Cases not covered

- `TransactionData::from_parts()` with nonzero ZIP 233 amount and `TxVersion::V5` - no existing tests.
- `TransactionData::from_parts()` or `from_parts_zfuture()` with nonempty TZE data and a non-zfuture version - no existing tests.
- Roundtrip tests that assert nonzero unsupported fields fail loudly instead of being silently dropped - no existing tests.
- `write_v5()` rejecting nonzero ZIP 233 amounts - no existing tests.

Attack surface

- `Transaction::read()` accepts untrusted transaction bytes from the network.
- `Transaction::write()` emits bytes that other consensus implementations will parse.
- Callers control the consensus branch ID passed into `Transaction::read()`.
- Version-specific feature fields such as ZIP 233, Ironwood, and TZE need to be handled carefully, because they may exist in memory only under compile flags but must still align with consensus wire format.

4.6. Component: Transaction IDs and Authorization Commitments

Description

This component is responsible for computing transaction identifiers, component digests, and block transaction authorization commitments. It is implemented in `zcash_primitives/src/transaction/txid.rs`.

It breaks a transaction into discrete parts. These are:

1. Header data
2. Transparent data

3. Sapling data
4. Orchard data
5. Ironwood data
6. TZE data

This component uses version-specific logic to decide which fields are included in a digest. For example, V6 includes Ironwood in the txid combination, while ZIP 233 is included in the header txid digest only when the version supports it. On the other hand, authorization data such as signatures and proofs are included in the authorization commitment instead of the txid.

Invariants

- Every non-witness field that has an effect on a transaction must be committed by the transaction ID.
- Authorization, proof, witness, and signature data must be committed by the authorization commitment where required.
- Empty Sapling, Orchard, Ironwood, and TZE bundles must use the protocol-defined empty digests.
- Ironwood must use a distinct bundle commitment domain from Orchard.
- The digest code must not include fields that are not present in the selected transaction version's wire format.
- Txid, sighash, and serialization must agree on which fields exist for each transaction version.

Test coverage

Cases covered

- ZIP 233 V6 paths for transaction ID, authorization commitment, and shielded sighash - has existing tests.
- V6 paths for Sapling, Orchard, and Ironwood anchor effects on txid and authorization commitment - has existing tests.
- V6 anchors affect authorization commitment but not txid or shielded sighash where intended - has existing tests.
- V5 behavior is different from V6 behavior - has existing tests.
- Empty bundle digest handling through txid computation - has existing tests.

Cases not covered

- Non-zero ZIP 233 amount paired with a version where `has_zip233()` is false - no existing tests.

Attack surface

- Wallets and signers rely on txid and sighash preimages to authorize the intended transaction.
- Compile flags and transaction versions control which fields exist, therefore inconsistent configurations or version gating can create txids that do not match consensus bytes.
- Low-level callers can pass inconsistent `TransactionData` to digest functions.
- Domain separation mistakes could make different pools or versions collide.

5. Assessment Results

During our assessment on the scoped librustzcash targets, we discovered five findings. No critical issues were found. Four findings were of low impact and the remaining finding was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.